Syntax-Native Incompleteness: A Chwistek-Inspired Formalization in Agda

Abstract

We present a fully constructive formalization of Gödel's first incompleteness theorem in Agda (3578 lines, 16 files) that uses binary trees as the native code representation, avoiding the traditional detour through arithmetic and Gödel numbering entirely. Self-reference is achieved via a finite quine construction in the style of Chwistek's metasystem/object-system separation. We develop two architectures—*stratified* and *fuel-based*—and prove Gödel I unconditionally in the stratified setting. A reflection hierarchy theorem shows that the stratified architecture *necessarily* separates meta-levels: each proof system can reflect the layer below but is provably blind to its own reflective reasoning. The fuel-based architecture collapses this hierarchy, enabling all three Hilbert–Bernays derivability conditions and a meta-level bounded Gödel II. The entire development uses no postulates, no standard library, no holes, and only ASCII identifiers; the formalization is self-contained and compiles with Agda's `--without-K --exact-split` flags.

1 Introduction

Gödel's incompleteness theorems are among the most celebrated results in mathematical logic, yet their formalizations typically follow a well-worn path: encode syntax as natural numbers via a Gödel numbering, develop significant arithmetic machinery (the β-function lemma, Σ_1-completeness), and then prove that the proof predicate is representable. This path is mathematically natural but computationally heavy and historically tied to the specific framework of Peano arithmetic.

Chwistek's approach to metamathematics [1] offers an alternative perspective. Rather than embedding syntax into arithmetic, Chwistek maintained a strict separation between the *object system* and the *metasystem*, manipulating expressions directly as syntactic objects. Guard [3] later explored a single-system approach with an internal proof predicate, defining $D_f(x) = Sb(x, n(x))$ to achieve self-reference by applying a formula to its own number.

In this paper, we pursue a *syntax-native* approach inspired by Chwistek's philosophy: syntax is represented as binary trees (type `Code`), not as natural numbers. Self-reference is achieved through a finite quine construction (`csub`) that decodes a formula from its tree code, substitutes, and re-encodes. The proof system is Hilbert-style with a decidable proof checker operating directly on tree codes. No arithmetic is needed.

Contributions.

(1) A complete Agda formalization of Gödel I with *no assumptions* (no consistency hypothesis, no soundness postulate) in the stratified architecture (§4).
(2) A *reflection hierarchy theorem* showing that the stratified evaluator is provably blind to extended proof codes, making the meta-level separation structurally necessary (§5).
(3) A fuel-based architecture where all three Hilbert–Bernays derivability conditions hold and a bounded Gödel II is proved at the meta-level (§6, §7).

(4) A sharp comparison of the two architectures, connecting them to Chwistek's stratified metamathematics and Guard's single-system approach (§8).

2 The Object Language

2.1 Code: Binary Trees as Syntax

The representation medium is the type of binary trees with natural-number leaves:

$$\mathsf{Code} ::= \mathsf{catom}(n) \mid \mathsf{cnode}(c_1, c_2) \qquad (n \in \mathbb{N})$$

Every syntactic category—variables, terms, formulas, schemas, proof trees—is encoded into Code via a tag scheme. For example, $\mathsf{catom}(3)$ encodes $\bot$, $\mathsf{cnode}(\mathsf{catom}(5), \mathsf{cnode}(\ulcorner A \urcorner, \ulcorner B \urcorner))$ encodes $A \to B$, and $\mathsf{cnode}(\mathsf{catom}(30), \ulcorner t \urcorner)$ encodes the `ax-refl` proof of $t = t$. Tags 0–9 are reserved for formulas, 10–13 for schemas, 14–19 for code expressions, and 30–36 for proof rules.

2.2 Terms and Formulas

Terms are built from de Bruijn–indexed variables, zero, and successor:

$$\mathsf{Term} ::= \mathsf{tvar}(v) \mid \mathsf{tzero} \mid \mathsf{tsucc}(t)$$

Formulas extend a minimal first-order language with code-level connectives:

$$\begin{aligned}\mathsf{Formula} ::= {} & \bot \mid \mathsf{feq}(s, t) \mid \mathsf{fimp}(A, B) \mid \mathsf{fall}(A) \\ & \mid \mathsf{fcode}(c) \mid \mathsf{fceq}(e_1, e_2) \mid \mathsf{fcAll}(A)\end{aligned}$$

Here $\mathsf{fcode}(c)$ embeds a code value as a formula, $\mathsf{fceq}(e_1, e_2)$ asserts that two code expressions evaluate to the same code, and $\mathsf{fcAll}(A)$ universally quantifies over code values (binding a code variable, distinct from the term variables bound by fall).

Code expressions are the computational fragment:

$$\mathsf{CExp} ::= \mathsf{cvar}(v) \mid \mathsf{clit}(c) \mid \mathsf{ccheck}(e) \mid \mathsf{csub}(e_1, e_2)$$

The expression $\mathsf{clit}(c)$ is a literal code value; $\mathsf{ccheck}(e)$ evaluates e to a proof code p, runs the proof checker, and returns the encoding of the proved formula; $\mathsf{csub}(e_1, e_2)$ decodes e_1 as a formula, substitutes $\mathsf{clit}(e_2)$ for code variable 0, and re-encodes. This last operation is the engine of the quine trick.

2.3 Proof System

The proof system is Hilbert-style with six constructors:

$$\begin{aligned}
&\mathsf{ax\text{-}refl}(t) : \mathsf{Proof}(\mathsf{feq}(t, t)) \\
&\mathsf{ax\text{-}k}(A, B) : \mathsf{Proof}(A \to (B \to A)) \\
&\mathsf{ax\text{-}s}(A, B, C) : \mathsf{Proof}((A \to B \to C) \to (A \to B) \to A \to C) \\
&\mathsf{mp}(p_1, p_2) : \mathsf{Proof}(A \to B) \to \mathsf{Proof}(A) \to \mathsf{Proof}(B) \\
&\mathsf{gen}(p) : \mathsf{Proof}(A) \to \mathsf{Proof}(\mathsf{fall} A) \\
&\mathsf{cgen}(p) : \mathsf{Proof}(A) \to \mathsf{Proof}(\mathsf{fcAll} A)
\end{aligned}$$

The checker function $\mathsf{checkProof} : \mathsf{Code} \to \mathsf{Maybe\ Formula}$ dispatches on tag numbers: tag 30 for `ax-refl`, 31 for `ax-k`, 32 for `ax-s`, 33 for `mp` (which recursively checks both subproofs), 34 for `gen`, and 35 for `cgen`. It is structurally recursive on Code and total.

2.4 Encoding and Decoding

The encoding functions $\mathsf{encFormula} : \mathsf{Formula} \to \mathsf{Code}$, $\mathsf{encTerm} : \mathsf{Term} \to \mathsf{Code}$, etc., are straightforward recursive mappings. The corresponding decoders $\mathsf{decFormula}$, $\mathsf{decTerm}$, etc., are partial (returning Maybe). The key roundtrip lemma is:

Lemma 2.1 (Roundtrip)**.** *For all* $A : \mathsf{Formula}$, $\mathsf{decFormula}(\mathsf{encFormula}(A)) = \mathsf{just}(A)$.

Analogous roundtrip lemmas hold for terms, code variables, code expressions, and schemas.

3 Self-Reference via the Quine Trick

3.1 Schemas and Diagonalization

A *schema* is a formula template with a single hole, to be filled by a code value:

$$\mathsf{Schema} ::= \mathsf{shole} \mid \mathsf{sconst}(A) \mid \mathsf{simp}(S_1, S_2) \mid \mathsf{sall}(S)$$

Instantiation $\mathsf{instantiate}(S, c)$ fills the hole with $\mathsf{fcode}(c)$, and encoding $\mathsf{encSchema}$ maps schemas to Code.

The *diagonal* of a schema feeds it its own code:

$$\mathsf{diag}(S) = \mathsf{instantiate}(S, \mathsf{encSchema}(S))$$

This is finite, terminating, and definitionally equal to its unfolding. It is the syntax-native analogue of Guard's $D_f(x) = Sb(x, n(x))$.

3.2 The Gödel Sentence

The Gödel sentence is constructed in three steps.

Step 1: The template φ. Define φ as a formula with one free code variable (index 0):

$$\varphi = \mathsf{fcAll}\big(\mathsf{fimp}\big(\mathsf{fceq}(\mathsf{ccheck}(\mathsf{cvar}_1),\ \mathsf{csub}(\mathsf{cvar}_0, \mathsf{cvar}_0)),\ \bot\big)\big)$$

Here cvar_0 will receive the self-code, and cvar_1 is bound by fcAll (it ranges over proof codes). Intuitively, φ says: "for all proof codes p, what p proves is not what csub computes from my code."

Step 2: The self-code. Let $\mathsf{phiCode} = \mathsf{encFormula}(\varphi) : \mathsf{Code}$.

Step 3: The Gödel sentence.

$$\mathcal{G} = \mathsf{substFormulaCode0}\big(\mathsf{clit}(\mathsf{phiCode}),\ \varphi\big)$$

Substituting $\mathsf{clit}(\mathsf{phiCode})$ for code variable 0 in φ yields:

$$\mathcal{G} = \mathsf{fcAll}\big(\mathsf{fimp}\big(\mathsf{fceq}(\mathsf{ccheck}(\mathsf{cvar}_0),\ \mathsf{csub}(\mathsf{clit}(\mathsf{phiCode}), \mathsf{clit}(\mathsf{phiCode}))),\ \bot\big)\big)$$

Theorem 3.1 (Self-Reference)**.** $\mathsf{evalCExp}(\mathsf{csub}(\mathsf{clit}(\mathsf{phiCode}), \mathsf{clit}(\mathsf{phiCode}))) = \mathsf{just}(\mathsf{encFormula}(\mathcal{G}))$.

Proof. By computation. The csub evaluator retrieves $\mathsf{phiCode}$, decodes it to φ (by the roundtrip lemma), substitutes $\mathsf{clit}(\mathsf{phiCode})$ for code variable 0, and re-encodes. The result is $\mathsf{encFormula}(\mathsf{substFormulaCode0}(c$ $\mathsf{encFormula}(\mathcal{G})$. □

Thus $\mathcal{G}$ says: "for all proof codes p, $\mathsf{checkProof}(p)$ does not equal $\mathsf{encFormula}(\mathcal{G})$"—i.e., "I am not provable." Crucially, $\mathcal{G}$ does *not* contain $\mathsf{encFormula}(\mathcal{G})$ as a subexpression (which would require an infinite term). It contains the finite expression $\mathsf{csub}(\mathsf{clit}(\mathsf{phiCode}), \mathsf{clit}(\mathsf{phiCode}))$, which *computes* $\mathsf{encFormula}(\mathcal{G})$ at evaluation time.

4 Gödel's First Incompleteness Theorem (Stratified)

4.1 Soundness

Truth is defined via an environment-based predicate $\mathsf{TrueF}(\rho, \sigma, A)$, structurally recursive on $\mathsf{Formula}$:

- $\mathsf{TrueF}(\rho, \sigma, \bot) = \varnothing$
- $\mathsf{TrueF}(\rho, \sigma, \mathsf{feq}(s, t)) = (\mathsf{eval}_\rho(s) \equiv \mathsf{eval}_\rho(t))$
- $\mathsf{TrueF}(\rho, \sigma, A \to B) = \mathsf{TrueF}(\rho, \sigma, A) \to \mathsf{TrueF}(\rho, \sigma, B)$
- $\mathsf{TrueF}(\rho, \sigma, \mathsf{fall}A) = \forall t.\ \mathsf{TrueF}(\rho[0 \mapsto t], \sigma, A)$
- $\mathsf{TrueF}(\rho, \sigma, \mathsf{fceq}(e_1, e_2)) = \exists c.\ \mathsf{eval}_\sigma(e_1) = \mathsf{just}(c) \wedge \mathsf{eval}_\sigma(e_2) = \mathsf{just}(c)$
- $\mathsf{TrueF}(\rho, \sigma, \mathsf{fcAll}A) = \forall c.\ \mathsf{TrueF}(\rho, \sigma[0 \mapsto c], A)$

A formula A is *true*, written $\mathsf{TrueFormula}(A)$, if $\mathsf{TrueF}(\rho, \sigma, A)$ holds for all environments ρ, σ.

Theorem 4.1 (Soundness)**.** *If* $\mathsf{Proof}(A)$*, then* $\mathsf{TrueFormula}(A)$*.*

Proof. By induction on $\mathsf{Proof}(A)$. Each case is direct: `ax-refl` gives refl; `ax-k` gives $\lambda a.\, \lambda_.\, a$; `ax-s` gives $\lambda f.\, \lambda g.\, \lambda a.\, f\, a\, (g\, a)$; `mp` applies the induction hypotheses; `gen` and `cgen` abstract over the new bound variable. □

4.2 Proof Encoding

Every Proof tree is encoded as a Code value by $\mathsf{encodeProof}$, and the checker accepts these codes:

Lemma 4.2 (Encoding Correctness)**.** *For all* $prf : \mathsf{Proof}(A)$*,* $\mathsf{checkProof}(\mathsf{encodeProof}(prf)) = \mathsf{just}(A)$*.*

4.3 The Main Theorem

Theorem 4.3 (Gödel I, No Assumptions)**.** $\mathsf{MetaNot}(\mathsf{Proof}(\mathcal{G}))$*, i.e., there is no proof of the Gödel sentence.*

Proof. Suppose $prf : \mathsf{Proof}(\mathcal{G})$.

1. By Theorem 4.1, $\mathsf{TrueF}(\rho_\varnothing, \sigma_\varnothing, \mathcal{G})$ holds.
2. Let $p = \mathsf{encodeProof}(prf)$. Since $\mathcal{G} = \mathsf{fcAll}(\ldots)$, instantiate the universal at $c = p$:
$$\mathsf{TrueF}(\rho_\varnothing, \sigma_\varnothing[0 \mapsto p], \mathsf{GoedelBody})$$
where $\mathsf{GoedelBody} = \mathsf{fimp}(\mathsf{fceq}(\mathsf{ccheck}(\mathsf{cvar}_0), \mathsf{csub}(\mathsf{clit}(\mathsf{phiCode}), \mathsf{clit}(\mathsf{phiCode}))), \bot)$. This means: if the two code expressions evaluate to the same value under $\sigma_\varnothing[0 \mapsto p]$, then $\varnothing$ (contradiction).
3. By Lemma 4.2, $\mathsf{checkProof}(p) = \mathsf{just}(\mathcal{G})$, so
$$\mathsf{eval}_{\sigma[0 \mapsto p]}(\mathsf{ccheck}(\mathsf{cvar}_0)) = \mathsf{just}(\mathsf{encFormula}(\mathcal{G})).$$
4. By Theorem 3.1,
$$\mathsf{eval}_{\sigma[0 \mapsto p]}(\mathsf{csub}(\mathsf{clit}(\mathsf{phiCode}), \mathsf{clit}(\mathsf{phiCode}))) = \mathsf{just}(\mathsf{encFormula}(\mathcal{G})).$$
5. Both sides evaluate to $\mathsf{encFormula}(\mathcal{G})$, so the code-equality witness exists. Applying step 2 yields $\varnothing$: contradiction. □

Remark 4.4. *No consistency hypothesis is assumed. The proof derives a contradiction in Agda's metatheory from the bare existence of a* $\mathsf{Proof}(\mathcal{G})$ *term. This is possible because soundness is proved* within *Agda for the specific (weak) object proof system, and the encoding correctness is verified computationally.*

5 The Reflection Hierarchy

5.1 The Extended Proof System

To approach Gödel II, we extend the proof system with a reflection rule:

$$\mathsf{ax\text{-}eval}(e, c, eq) : \mathsf{ProofExt}(\mathsf{fceq}(e, \mathsf{clit}(c))) \qquad \text{when } \mathsf{evalCExp}(e) = \mathsf{just}(c)$$

This rule internalizes evaluation results: if a closed code expression e evaluates to c, the extended system can prove the corresponding fceq formula. The extended checker checkProofExt handles all tags 30–36, where tag 36 is `ax-eval`.

The first derivability condition holds for base proofs:

Proposition 5.1 (D1 for Base Proofs)**.** *For any prf* : $\mathsf{Proof}(A)$,

$$\mathsf{ProofExt}\big(\mathsf{fceq}(\mathsf{ccheck}(\mathsf{clit}(\mathsf{encodeProof}(prf))),\ \mathsf{clit}(\mathsf{encFormula}(A)))\big).$$

5.2 The Blindness Lemma

The stratified architecture creates an asymmetry: the evaluator evalCExp calls the *old* checker checkProof (tags 30–35), not the extended checkProofExt (tags 30–36). Consequently:

Lemma 5.2 (Blindness)**.** *For any* `ax-eval` *proof prf* $= \mathsf{ax\text{-}eval}(e, c, eq)$,

$$\mathsf{evalCExp}(\mathsf{ccheck}(\mathsf{clit}(\mathsf{encodeProofExt}(prf)))) = \mathsf{nothing}.$$

Proof. The encoding $\mathsf{encodeProofExt}(\mathsf{ax\text{-}eval}(e, c, eq))$ has tag 36. The old checkProof dispatches through tags 30–35 and falls through to nothing. This holds *definitionally* (by `refl` in Agda). □

Theorem 5.3 (No Self-Reflection)**.** *For any* `ax-eval` *proof prf, there is no code d such that* $\mathsf{evalCExp}(\mathsf{ccheck}(\mathsf{clit}(\mathsf{encodeProofExt}(prf)))) = \mathsf{just}(d)$.

Proof. By the blindness lemma, the left-hand side is nothing, and $\mathsf{nothing} \neq \mathsf{just}(d)$ for any d. □

5.3 Strictness of the Hierarchy

Combining these results:

- **Positive:** For any old $\mathsf{Proof}(A)$, the extended system can internally state that the proof code checks to A.
- **Negative:** For any `ax-eval` proof, the extended system *cannot* state the same thing using `ax-eval`, because the old evaluator is blind to tag 36.

The pattern repeats: a second extension reflecting checkProofExt would be blind to its own tag, requiring a third extension, and so on. Each consistency or completeness result requires a strictly stronger metasystem—exactly as Chwistek's framework predicts.

Corollary 5.4. *D3 (the third Hilbert–Bernays derivability condition) is* impossible *in the stratified architecture. Consequently, Gödel II cannot be proved in this setting.*

6 The Fuel-Based Architecture

6.1 Unified Mutual Recursion

To overcome the D3 barrier, we replace the stratified evaluator/checker pair with a *fuel-based* mutual recursion:

$$\mathsf{checkProofN} : \mathbb{N} \to \mathsf{Code} \to \mathsf{Maybe\ Formula}$$
$$\mathsf{evalCExpN} : \mathbb{N} \to \mathsf{CExp} \to \mathsf{Maybe\ Code}$$

Both functions decrease the fuel parameter at each mutual call, ensuring termination. Crucially, evalCExpN's ccheck case calls checkProofN (which handles tag 36), not the old checkProof. Tag 36 in checkProofN calls evalCExpN to verify the evaluation result at runtime, guarded by a Boolean equality check on the code.

6.2 All Three Derivability Conditions

Theorem 6.1 (D1, Fuel-Based)**.** *Given* $\mathsf{checkProofN}(n{+}1, p) = \mathsf{just}(A)$*, there exists a code* q *with*

$$\mathsf{checkProofN}(n{+}4, q) = \mathsf{just}\big(\mathsf{fceq}(\mathsf{ccheck}(\mathsf{clit}(p)),\ \mathsf{clit}(\mathsf{encFormula}(A)))\big).$$

Each level of representation costs 3 additional fuel.

Proof. The witness $q = \mathsf{cnode}(\mathsf{catom}(36), \mathsf{cnode}(\mathsf{encCExp}(\mathsf{ccheck}(\mathsf{clit}(p))), \mathsf{encFormula}(A)))$. The proof chains through: (1) decoding the CExp encoding (by the roundtrip lemma), (2) verifying that evalCExpN at the appropriate fuel yields $\mathsf{encFormula}(A)$ (by the hypothesis), and (3) reflexivity of code equality. □

D2 (closure under modus ponens at the code level) follows by the same tag-33 construction as in the base system. D3 is simply D1 applied twice:

Theorem 6.2 (D3, Fuel-Based)**.** *Given* $\mathsf{checkProofN}(n{+}1, p) = \mathsf{just}(A)$*, there exists a code* q' *accepted at fuel* $n{+}6$ *as a proof that the D1 representation is itself represented.*

Proof. Apply Theorem 6.1 to the D1 witness itself. □

The hierarchy has *collapsed*: unlike the stratified architecture, where the evaluator was blind to tag 36, the fuel-based checkProofN handles tag 36 at every fuel level. Self-reflection works because the checker sees its own reflected proofs.

7 Bounded Gödel II

7.1 Fuel-Based Soundness

The fuel-based proof type $\mathsf{ProofN}(n)$ parameterizes proofs by fuel level. It includes all base constructors plus $\mathsf{axEvalN}(e, c, eq)$ when $\mathsf{evalCExpN}(n, e) = \mathsf{just}(c)$.

Theorem 7.1 (Soundness for ProofN)**.** *If* $\mathsf{ProofN}(n, A)$*, then* $\mathsf{TrueFormulaN}(n, A)$*, where* TrueFormulaN *is the truth predicate using* evalCExpN *at fuel* n*.*

7.2 Bounded Gödel I

Theorem 7.2 (Bounded Gödel I)**.** *If* $prf : \mathsf{ProofN}(n{+}3, \mathcal{G})$ *and* $\mathsf{checkProofN}(n{+}2, \mathsf{encodeProofN}(prf)) = \mathsf{just}(\mathcal{G})$*, then* $\varnothing$ *(Empty).*

Proof. The same structure as Theorem 4.3: soundness gives truth of $\mathcal{G}$, the encoding gives a proof code, and the self-reference property (which holds for fuel ≥ 2 since csub of literals does not invoke the checker) produces the contradiction. □

The encoding correctness assumption ($\mathsf{checkProofN}(n{+}2, \mathsf{encodeProofN}(prf)) = \mathsf{just}(\mathcal{G})$) is the fuel-based analogue of Lemma 4.2. In the stratified setting, this was proved by induction on the proof tree. In the fuel-based setting, the same proof would go through but requires adapting the entire encoding correctness chain, which is left as the `enc-correct` parameter.

7.3 Meta-Level Bounded Gödel II

Bounded consistency is expressed as: no proof code checked at fuel n proves $\bot$. The consistency formula Con is:

$$\mathsf{Con} = \mathsf{fcAll}\big(\mathsf{fimp}\big(\mathsf{fceq}(\mathsf{ccheck}(\mathsf{cvar}_0),\ \mathsf{clit}(\mathsf{encFormula}(\bot))),\ \bot\big)\big)$$

Theorem 7.3 (Meta-Level Bounded Gödel II)**.** *If* $\mathsf{ProofN}(n{+}3, \mathcal{G})$ *with encoding correctness, then* $\varnothing$*. Hence* $\mathsf{ProofN}(n{+}3, \mathsf{Con})$ *and* $\mathsf{ProofN}(n{+}3, \mathcal{G})$ *(with encoding correctness) cannot both exist.*

The full *internal* Gödel II—deriving $\mathsf{ProofN}(m, \mathsf{fimp}(\mathsf{Con}, \mathcal{G}))$—requires the system to prove its own soundness internally, which is the classical barrier. The fuel-based architecture provides all the necessary derivability conditions (D1, D2, D3), but the final step of internalizing soundness remains a universal limitation, not an architectural one.

8 Comparison: Chwistek, Guard, and the Two Architectures

8.1 Chwistek's Stratified Metamathematics

Chwistek [1] insisted on a strict separation between object system and metasystem, manipulating expressions directly rather than through arithmetic encoding. Our stratified architecture embodies this philosophy: evalCExp calls checkProof (the object-level checker), and checkProofExt lives at a higher meta-level. The blindness lemma (Lemma 5.2) *proves* that this separation is not merely presentational but structurally necessary.

8.2 Guard's Single-System Approach

Guard [3] pursued a single-system approach with an internal proof predicate, defining self-reference via $D_f(x) = Sb(x, n(x))$. Our fuel-based architecture corresponds to this: checkProofN and evalCExpN form a single mutual recursion where the checker can see its own reflected proofs. The derivability conditions all hold, but at the cost of a fuel parameter that bounds the depth of self-reflection.

	Stratified	**Fuel-Based**
Gödel I	Proved (no assumptions)	Proved (with `enc-correct`)
D1	Yes (base proofs only)	Yes (all proofs)
D3	Blocked (blind to tag 36)	Works (+3 fuel per level)
Hierarchy	Strict (proved)	Collapses
Gödel II	Impossible	Meta-level proved

Table 1: Comparison of the two architectures.

8.3 Sharp Comparison

The stratified architecture gives a *stronger* Gödel I (no assumptions at all) and a richer structural theorem (the hierarchy), but cannot reach Gödel II. The fuel-based architecture gives weaker individual theorems (encoding correctness is assumed) but provides the full derivability infrastructure needed for Gödel II.

9 Related Work

Formalizations of the incompleteness theorems in proof assistants include the work of Shankar [7] in Nqthm, O'Connor's formalization in Coq [5], and Paulson's formalization of the incompleteness theorems in Isabelle/HOL [6]. All of these follow the arithmetic route, encoding syntax into natural numbers and developing substantial number-theoretic infrastructure.

Our approach is closer in spirit to the "syntax-based" formalizations explored by Rathjen and Leigh [4] and to the category of "direct" or "semantic" proofs of incompleteness, where the diagonal lemma is applied at the level of syntax rather than arithmetic. The key novelty is the use of binary trees as the native representation, which eliminates the need for Gödel numbering entirely and makes the self-reference construction (via csub) both finite and transparent.

The reflection hierarchy theorem connects to the well-known phenomenon of ordinal analysis and proof-theoretic strength: each consistency statement requires a strictly stronger system. Our formalization makes this structural necessity explicit at the level of a concrete proof checker.

10 Conclusion

We have presented a fully constructive, self-contained formalization of Gödel's first incompleteness theorem in Agda, using binary trees as the native syntax representation and avoiding Gödel numbering entirely. The development comprises 16 files and 3578 lines of Agda, with no postulates, no holes, and no standard library.

The key technical insight is that self-reference can be achieved through a finite quine construction (csub) operating on tree codes, without any detour through arithmetic. The formalization revealed a sharp structural dichotomy between stratified and fuel-based architectures: the stratified approach gives an unconditional Gödel I and a strict reflection hierarchy, while the fuel-based approach enables all three derivability conditions and a bounded Gödel II.

The reflection hierarchy theorem—showing that each proof system can reflect the layer below but is provably blind to its own reflective reasoning—provides formal content to Chwistek's insistence on the necessity of metasystem/object-system separation. In the fuel-based architecture, this hierarchy collapses, aligning with Guard's single-system approach, but at the cost of a fuel bound and an encoding correctness assumption.

The remaining gap for a fully internal Gödel II is the same gap that exists in any sufficiently strong system: the system cannot prove its own soundness. This is not a limitation of the architecture but the very content of Gödel's second incompleteness theorem.

Artifact. The complete Agda development is available as 16 source files. All files compile with Agda's `--without-K --exact-split` flags. The `--safe` flag is used throughout; no postulates or unsafe pragmas appear anywhere in the development.

References

[1] L. Chwistek. *The Limits of Science: Outline of Logic and of the Methodology of the Exact Sciences*. Kegan Paul, Trench, Trubner & Co., London, 1935/1948. (Original Polish edition: *Granice nauki*, 1935.)

[2] K. Gödel. Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I. *Monatshefte für Mathematik und Physik*, 38:173–198, 1931.

[3] J. R. Guard. The equivalence of two formulations of Gödel's theorem. *Notre Dame Journal of Formal Logic*, 10(3):235–244, 1969.

[4] G. E. Leigh and M. Rathjen. The Friedman–Sheard programme in intuitionistic logic. *Journal of Symbolic Logic*, 77(3):777–806, 2012.

[5] R. O'Connor. Essential incompleteness of arithmetic verified by Coq. In *Theorem Proving in Higher Order Logics (TPHOLs 2005)*, LNCS 3603, pages 245–260. Springer, 2005.

[6] L. C. Paulson. A mechanised proof of Gödel's incompleteness theorems using Nominal Isabelle. *Journal of Automated Reasoning*, 55(1):1–37, 2015.

[7] N. Shankar. *Metamathematics, Machines, and Gödel's Proof*. Cambridge Tracts in Theoretical Computer Science 38. Cambridge University Press, 1994.